1BM120 - Assignment 3

Surrogate-based optimization, multi-objective optimization

By dr. ir. Laurens Bliek
30-01-2026

20 points

Problem description

TU/e and TNO are part of a project to build a large off-shore wind farm on the North Sea. A wind farm consists of multiple wind turbines that are often placed according to a grid or some other layout. However, due to wind turbulence, if the wind conditions are extreme (uncommon wind angle and large wind speed), turbulence can cause certain wind turbines to produce less energy. By placing wind turbines according to a different layout than the standard one, the average energy output can be increased.

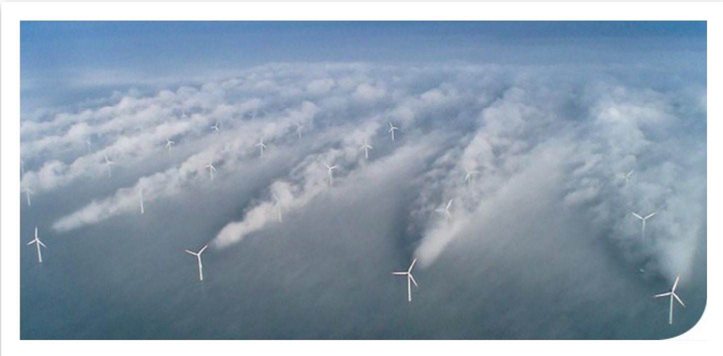

Figure 1: Extreme wind conditions can cause turbulence, which can cause wind turbines to produce less energy.

In this assignment you will use everything you have learned in this course to search for the optimal wind farm layout by placing five wind turbines in an assigned area, generating as much renewable energy as possible. While this task typically requires wind simulators that take seconds, hours or even weeks for their computations, you will be using a **surrogate model** that makes predictions much faster.

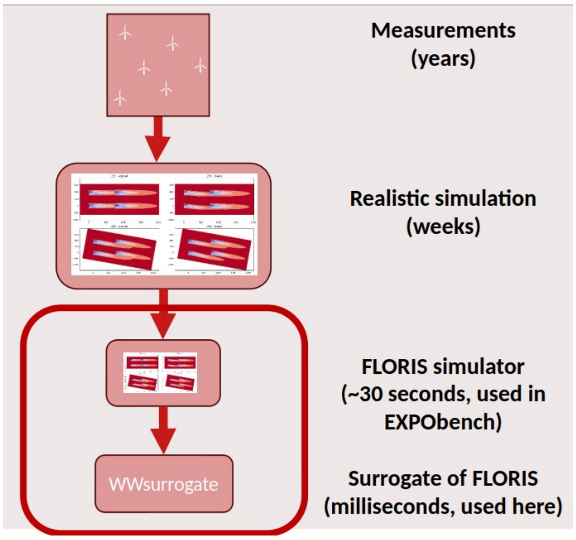

Figure 2: Multiple fidelities of wind farm energy measurements

The problem can be stated as a minimization problem:

$$\underset{x \in X}{\operatorname{argmin}} f(x)$$
$$X = [0,1]^{10} \text{ (for 5 turbines)}$$

Here, f is -1 times the average energy output in GWh produced by the wind farm on a yearly basis under random wind conditions. An optimally designed wind farm with five wind turbines might be able to produce about 70 GWh per year, or even more. The variables [x1, x2, ..., x10] denote the wind turbine locations, where [x1,x2] are the horizontal and vertical coordinates respectively of the first wind turbine, and [x9,x10] are the horizontal and vertical coordinates of the last wind turbine. The wind farm is a square area in the North Sea, of size 1666.65x1666.65 meters, scaled to a 1x1 square in the formula above. The wind turbines have a rotor diameter of 126 meters.

The surrogate model *f* is an ensemble of five XGBoost machine learning models of different sizes that are trained on data from a wind simulator called FLORIS. The ensemble produces a noisy output, where the noise variance is larger if the different XGBoost models disagree on the output, and noise is low if the different models agree. The objective f is called *objective1* in the given code.

Do not make any changes inside the objective function in the given code – you should treat the surrogate model as a black box.

The same holds for the other functions in the given code. You can change what goes into and out of the black box (e.g. transform the x variables or the search space, post-process the output, etc.).

Overall assignment

Your goal is to solve the optimization problem above, for four different scenarios A,B,C,D. For each scenario, you have to compare three different algorithms:

- Random search
- One algorithm that was taught in the course, with minimal adaptations
- One algorithm that goes beyond what was taught in the course, with adaptations either invented by your group (state clearly your thought process) or found in the scientific literature (use clear citations)

The algorithms can be changed for each scenario.

Make sure to run each of the three algorithms at least 5 times, so that an average result can be calculated as well as standard deviations. In addition, show the following:

- Statistical tests on the best found solution that analyse whether one algorithm is better than another algorithm
- Convergence plots of the best found solution so far, that show how the algorithms manage to find better solutions over time
- A visualization and analysis of the best found solutions
- Motivations for why you choose to implement a certain algorithm and why you make certain adaptations
- Computation times for all algorithms and the number of times each algorithm evaluated the objectives and/or constraints

Scenario A: expensive optimization

In this scenario, we pretend that the optimization problem is more expensive than is the case for the surrogate model. Every time *objective1* is evaluated, you have to pretend that 30 seconds of computation time are needed. In this scenario, you have a **fixed computational budget** available of at most 5 hours for both evaluating this expensive objective and for running your algorithm. The budget applies separately for each run of each algorithm.

Although you could let your code wait for 30 seconds every time the objective is evaluated, this will be very slow. The easiest way to deal with this scenario is to do post-processing: after the algorithm is finished, simply add 30 seconds per evaluation to the computation time.

Example 1:

You run random search five times, for 400 iterations. Since at each iteration, you evaluate *objective1* once, this means that for each of the five runs, it takes 200 minutes to do all the evaluations. Together with the time spent on random search, you take less than 201 minutes, which is well within the budget. You repeat this five times.

Example 2:

You run an advanced optimization algorithm five times, and in each run it also evaluates *objective1* 400 times. It still takes 200 minutes to do all the evaluations for one run. However, the algorithm itself also happens to take a long time, 500 minutes in total for one run. In total, this is 700 minutes per run, which is more than the computational budget of 5 hours, and not allowed. You would have to decrease the number of iterations or otherwise make sure that the algorithm is more efficient.

Scenario B: constrained optimization

In this scenario, a **constraint** is added to the problem. For safety reasons, two wind turbines are not allowed to be too close to each other. The minimum distance between any two wind turbines is 2 times their diameter. This constraint is implemented in the function *constraint1*. It returns 1 if the constraint is satisfied, and 0 if the constraint is not satisfied. There is no computational budget in this scenario, but the final solution for each run of each algorithm must satisfy the constraint.

Scenario C: multi-objective optimization

In this scenario, we have **multiple objectives**. Besides generating as much renewable energy as possible, we also have an ecological factor to take into account. The wind farm will be placed right next to a bird corridor (an area with no human-made constructions, to allow birds to migrate), and depending on where the wind turbines are placed, this corridor can be made wider or narrower, influencing biodiversity.

The second objective, a biodiversity metric that is also adapted for minimization, can be found in *objective2*. A smaller value indicates a wider bird corridor that lets more birds through the area: 0 corresponds to having no wind farm at all, leaving the most space for the birds with a wide corridor, and 1 corresponds to the narrowest bird corridor that is still allowed. The objective is calculated by sampling birds from a probability distribution and counting how many get too close to the left-most wind turbine when flying from North to South. The wind farm is located just East of the bird corridor. Your task is to find the Pareto front for minimizing both objectives, *objective1* and *objective2*. There is no easy way to weigh these objectives against each other, hence the need for finding the full Pareto front.

Instead of statistical tests and convergence plots, please analyze the Pareto fronts found by the different algorithms in this scenario. The main question to be answered is how much wider the bird corridor can become without losing too much in terms of renewable energy generation. There is no computational budget or constraint.

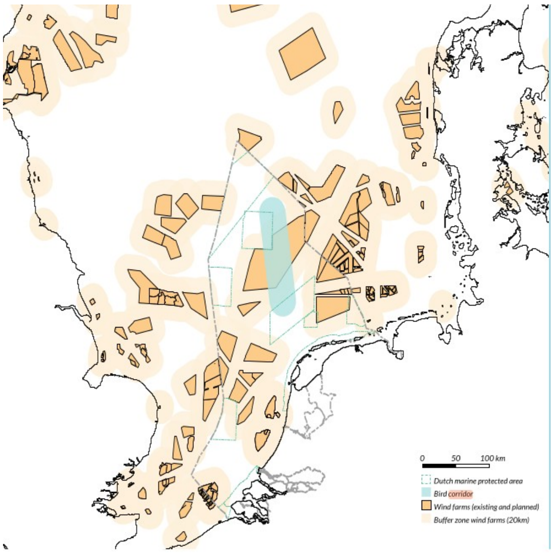

Figure 2: A bird corridor allows birds and other animals to migrate from North to South and vice versa.

Scenario D: bonus

This is a **bonus** scenario: your grade will not be affected if you skip this one. Since this is an active research area, if you solve it well, you might just have what it takes to become a researcher at TU/e or TNO or a similar organization!

Scenario D is a combination of scenarios A, B, and C. Two objectives, one constraint, and a computational budget are all in effect at the same time, highly complicating the problem. Can you find the Pareto front for this situation?

Required packages

The following packages are required to run the objectives and constraint that are found in the Jupyter notebook *1BM120_assignment3.ipynb*:

```
numpy pickle xgboost scipy
```

These can be installed with a package manager or *pip install*.

You also need the file *Ensemble.pkl* to be in the same folder as the Jupyter notebook.

The code in *objective1* might give a warning about an older version of XGBoost, but if you run the code a second time and the warning disappears then everything is fine.

If you get the following error:

ModuleNotFoundError: No module named 'pkg_resources'
try the following code first:
pip install --upgrade setuptools

More specifications on the wind farm can be found in
https://github.com/NREL/floris/blob/v2/examples/example_input.json

Submission details

The project must be implemented using Python 3.7+. The submission folder of the project should contain the following files:

- Python file, containing:
  - code for running all the algorithms in the different scenarios (1 point per scenario).
  - code for the analysis and visualization (1 point per scenario)
  - (code for both can be either in python files or Jupyter notebook files)
- Report file, containing:
  - An explanation for how each scenario is solved (which algorithm, which adaptations), with motivation and with clear links to either the course material or scientific literature, or a thought process for original ideas. (1 point per scenario)
  - Convergence plots of the best found solution so far, averaged over multiple runs, separated by algorithm, for scenarios A and B. (1 point per scenario)
  - Pareto fronts for scenarios C (1 point) and D (1 bonus point)
  - Statistical tests on the best found solution for each algorithm, for scenarios A and B (1 point per scenario).
  - Visualization and analysis of the best found solutions (1 point per scenario)
  - Computation times and number of objective/constraint evaluations for all algorithms (1 point per scenario).

Scenario D can give a maximum of 3 bonus points total (including the Pareto front).

The files described above must be submitted in a zip file. The **maximum length** of the report is 8 pages, excluding a title page and references.

The rubric appears on the next page. Good luck with the assignment!

Rubric

Task	Description
Code for running all algorithms (1 point per scenario)	The code is functional and well-documented for each algorithm and each scenario.
Code for the analysis and visualization (1 point per scenario)	The code is functional and well-documented for analysis and visualization of the solutions.
Explanations and motivations (1 point per scenario)	The report describes clearly which algorithm or adaptation was chosen for each scenario, with a clear motivation, and a clear link to the course material, or thought process, or reference to the scientific literature. Choices are well motivated and suitable to the problem at hand, and original ideas are novel and creative.
Convergence plots (1 point per scenario, A and B only)	The best found solution so far is provided in a clear convergence plot, averaged over multiple runs, separated by algorithm, for scenarios A and B. The plots are explained in the report.
Pareto fronts (1 point for C, 1 bonus point for D)	Pareto fronts are clearly visualized for the different algorithms.
Statistical tests (1 point per scenario, A and B only)	The proper statistical tests are used to investigate whether one algorithm is better than the other for a certain scenario, using multiple runs.
Visualization and analysis (1 point per scenario)	The best found solutions are visualized as wind farm layouts. These are analyzed in the report using terminology and units that are understandable for a decision-maker.
Computation times and evaluations (1 point per scenario)	Computation times for all algorithms are mentioned in the report, as well as the number of objective/constraint evaluations. In Scenario A these are analyzed in more detail as they play a larger role.
Scenario D (up to 3 bonus points including the Pareto front)	For bonus points, Scenario D can be treated as the other scenarios.